

PythonIntermediateCourse-es007

June 11, 2026

1 Curso básico de Python

1.1 Apuntes intermedio

Curso básico de Python. Apuntes intermedio por Marcelo Horacio Fortino. Versión 2.5.1. Junio 2026.

Esta obra está sujeta a la licencia Reconocimiento-CompartirIgual 4.0 Internacional de Creative Commons. Para ver una copia de esta licencia, visite <http://creativecommons.org/licenses/by-sa/4.0/>. Puede hallar permisos más allá de los concedidos con esta licencia en <https://fortinux.com>. Sugerencias y comentarios a info@fortinux.com.

Todas las marcas son propiedad de sus respectivos dueños. Python® y PyCon® son marcas registradas de la Python Software Foundation. Linux® es una marca registrada de Linus Torvalds. Ubuntu® es una marca registrada de Canonical Limited. Google® es una marca registrada de Google Inc. Microsoft® y Windows® son marcas registradas de Microsoft Corporation. Red Hat® es una marca comercial registradas de Red Hat, Inc. UNIX® es una marca comercial registrada de The Open Group. Apple® y Mac OS® son marcas comerciales de Apple Inc.

Versión	Autor/es	Fecha	Observaciones
1.0	Marcelo Horacio Fortino	2021/Marzo	Curso Python
1.1	Marcelo Horacio Fortino	2021/Junio	Convertido a markdown - ipynb
1.2	Marcelo Horacio Fortino	2021/Agosto	Actualizados contenidos
1.3	Marcelo Horacio Fortino	2021/Octubre	Agregado Flask microframework
1.4	Marcelo Horacio Fortino	2021/Noviembre	Agregado Pandas - datascience
1.5	Marcelo Horacio Fortino	2021/Diciembre	Agregado Devops - Ansible
2.0	Marcelo Horacio Fortino	2022/Abril	Nueva estructura: core / module
2.1	Marcelo Horacio Fortino	2022/Junio	Módulo apuntes intermedio
2.2	Marcelo Horacio Fortino	2022/Agosto	Actualizado temario y ejercicios

Versión	Autor/es	Fecha	Observaciones
2.3	Marcelo Horacio Fortino	2022/Octubre	Actualizado despliegue a render.com
2.4	Marcelo Horacio Fortino	2023/Noviembre	Aplicaciones de escritorio con python
2.5	Marcelo Horacio Fortino	2026/Abril	Aplicaciones de ML e IA para python

Esta obra se distribuye con la esperanza de que sea útil, pero SIN NINGUNA GARANTÍA, incluso sin la garantía MERCANTIL implícita o sin garantizar la CONVENIENCIA PARA UN PROPÓSITO PARTICULAR. El autor no asume ninguna responsabilidad si el lector hace un mal uso de la misma.

Estos apuntes se basan en: - La documentación oficial de Python, <https://docs.python.org/es/3/tutorial/index.html>, - La bibliografía presentada al final de este documento, y - Documentación propia recogida a lo largo de los años de diversas fuentes.

1.2 Objetivos del curso

- Objetivo general del curso:
- Aprender a usar Python para crear scripts y programas simples plenamente funcionales, desarrollar aplicaciones web, realizar análisis de datos, y utilizar bibliotecas de inteligencia artificial (AI) para aprendizaje automático (ML).
- Objetivos específicos:
- Reconocer las características principales de Python y su utilidad práctica.
- Identificar tipos de datos (simples y compuestos) y operadores.
- Aplicar variables y estructuras de control de flujo.
- Construir funciones y clases (POO).
- Clasificar los módulos y paquetes por sus funcionalidades y objetivos.
- Comparar el lenguaje con otros similares de scripts, procedimentales y orientados a objetos.
- Probar bibliotecas para conexiones REST a aplicaciones web y bases de datos.
- Resolver problemas y errores en el código fuente proponiendo soluciones alternativas (refactoring).
- Utilizar la biblioteca pandas junto con matplotlib y numpy para realizar análisis estadísticos de datos y gráficos.
- Descubrir las herramientas en python de ML y DL (*Deep Learning*) para la inteligencia artificial.
- Como resultado práctico al final del curso cada estudiante habrá creado una aplicación web utilizando el microframework de Python Flask.

1.3 Temario

- Introducción, instalación y compilación
- Datos, expresiones y sentencias
- Variables y funciones, control de flujo

- Clases y objetos, herencia, polimorfismo
- Entradas y salidas con Python
- Gestión de módulos, paquetes y bibliotecas
- Servicios y programas en red, REST API
- Desarrollo de aplicaciones web con Flask
- Análisis de datos con pandas, matplotlib y numpy
- Aprendizaje automático (ML) e inteligencia artificial (AI) con Python
- Módulos opcionales
 - DevOps con Ansible
 - Plotting con Python
 - SDK de GCP para Python
 - SQL ejemplos con pandas
 - Docker y kubernetes con Python
 - Aplicaciones de escritorio con python

1.4 Bibliografía

- Downey, A., Elkner, J., Meyers, C. (2015). Aprenda a Pensar Como un Programador con Python.
Recuperado de <https://argentinaenpython.com/quiero-aprender-python/aprenda-a-pensar-como-un-programador-con-python.pdf>
- Kent D. Lee. (2014). Python, Programming Fundamentals Second Edition.
- Marzal Varó, A., Gracia Luengo, I., García Sevilla, Pedro. (2014). Introducción a la programación con Python 3.
Recuperado de <http://repositori.uji.es/xmlui/handle/10234/102653>
- Miller, B., Ranum, D. Solución de problemas con algoritmos y estructuras de datos usando Python. Traducido por Mauricio Orozco-Alzate, Universidad Nacional de Colombia - Sede Manizales.
Recuperado de <https://runestone.academy/ns/books/published/pythoned/index.html?mode=browsing>
- Shaw, Z. A. (2016). Learn Python 3 the Hard Way.
Recuperado de <https://learnpythonthehardway.org/>
- Van Rossum, G. and the Python development team. (2020). Documentación de Python en español.
Recuperado de https://python-docs-es.readthedocs.io/_/downloads/es/pdf/pdf/
- McGregor, S. E. (2022). Practical Python data wrangling and data quality. O'Reilly Media.
- Chollet, F. (2017). Deep learning with Python. Manning Publications.
- Russell, S., & Norvig, P. (2019). Artificial intelligence: A modern approach. Pearson Higher Education.
- D2L.ai: Interactive Deep Learning Book with Multi-Framework Code, Math, and Discussions.
Recuperado de <https://d2l.ai/>

2 Servicios y programas en red

- Manipular ficheros, Analizador sintáctico - parser, Formato JSON, Objetos en Python y su equivalente en JSON, Persistencia de datos, REST API, Herramientas útiles, Formato YAML, Persistencia de datos, Manipular ficheros XML, Seguridad XML, Descargar ficheros desde Internet, Biblioteca Requests, BeautifulSoup, Scrapy, Selenium.
- Errores y excepciones en Python: Errores de sintaxis, Excepciones, Depuración de errores, Logging, Formatos de docstrings, Doctests, Pruebas unitarias, Argparse.

2.1 Manipular ficheros

2.2 Analizador sintáctico - parser

- En el mundo de la programación *parsing* es el proceso de analizar una cadena de texto identificando su estructura sintáctica para poder convertir esos datos a un formato en el cual se pueda trabajar con ellos:
 - Dividir cadenas de texto o separarlas con comas o tabuladores.
 - Cambiar el formato a un fichero, convertirlo en binario, etc.
- Un ejemplo básico sería convertir un fichero con formato *.csv* a *.ods* para poder trabajar con un programa de ofimática (hoja de cálculo en este caso).
- Según la Wikipedia *parser* se puede traducir como analizador sintáctico:
- https://es.wikipedia.org/wiki/Analizador_sint%C3%A1ctico.
- En Python tenemos tres posibilidades: usar una biblioteca, desarrollar un *parser* propio, o utilizar una herramienta que genere un *parser*.
- En el primer caso, existen bibliotecas para los formatos de ficheros más conocidos como, XML, HTML, CSV, etc.
- El segundo caso es cuando se necesita mejorar el desempeño, cuando se desea integrar distintos componentes del sistema, o cuando no existe un *parser* para un formato específico de ficheros.
- El tercer caso finalmente sería una vía de medio para los otros dos.
- En los siguientes apartados veremos como analizar ficheros con formatos JSON, XML, y obtener datos de ficheros con logs.
- Tutoriales con ejemplos: <https://www.vipinajayakumar.com/parsing-text-with-python/> y <https://tomassetti.me/parsing-in-python/>.

2.3 Formato JSON

- *JSON (JavaScript Object Notation)* es un formato popular utilizado para representar datos estructurados.
- En Python se utiliza comúnmente para transmitir y recibir datos entre una aplicación y un servidor web.
- El módulo para este fin se denomina *json*.
- El módulo *json* codifica objetos de Python como cadenas JSON y decodifica las cadenas JSON convirtiéndolas en objetos de Python:
 - *json.dumps()*: Obtiene un objeto de Python y lo convierte (*dumps*) a una cadena (*string*).
 - *json.loads()*: Obtiene una cadena JSON y la convierte (*loads*) a un objeto de Python.

```
[ ]: import json
```

```
# Ejemplo de diccionario
persona = {"nombre": "Juan", "lenguajes": ["Python", "Shellscripts"]}
print(persona)
print("Tipo:", type(persona))
```

```
[ ]: # Ejemplo de string en JSON
persona = '{"nombre": "Juan", "lenguajes": ["Python", "Shellscripts"]}'
print(persona)
print("Tipo:", type(persona))
```

```
[ ]: # Parsing usando el método json.loads()
# Crea un diccionario en python
persona_dic = json.loads(persona)
print(persona_dic['lenguajes'])
print("Tipo:", type(persona_dic))
```

```
[ ]: # Convierte diccionario a JSON
# .dumps (encode) toma un diccionario como input y devuelve un string
persona_json = json.dumps(persona_dic)
print(persona_json)
print("Tipo:", type(persona_json))
```

- Para leer un fichero *.json* y luego exportarlo:

```
[ ]: import json

with open('ejemplo_json_fichero.json') as fichero:
    # loads (decode) toma un string como input y devuelve un diccionario
    datos = json.loads(fichero.read())
    print(datos)

# Lee solo un dato del fichero
print(datos['Nombre'])

# Exporta la variable datos a un fichero .json
with open('ejemplo_persona_json.txt', 'w') as json_fichero:
    json.dump(datos, json_fichero)
```

2.4 Objetos en Python y su equivalente en JSON

Python	JSON
dict	object

Python	JSON
list, tuple	array
str	string
int, float, int	number
True	true
False	false
None	null

- Fuente: <https://rico-schmidt.name/pymotw-3/json/index.html>.

2.5 REST API

- *REST (REpresentational State Transfer Application Programming Interface)* o API de *RESTful* es en esencia una serie de convenciones que sirven para estructurar una API que interactúa utilizando el protocolo HTTP.
- Permite hacer pedidos a una URL mediante dos métodos: GET, que obtiene datos, y POST, que los envía.
- El ejemplo clásico es obtener un formulario (GET) de una página web y una vez relleno, enviarlo (POST).
- Otros métodos son PUT (reemplaza datos) y DELETE (borra datos), aunque hay más.
- Python cuenta con la biblioteca *urllib.request* que define funciones y clases para:
 - apertura de URLs mediante HTTP, autenticación básica y digest, redirecciones, cookies y más.
- El listado de todos los métodos existentes se puede consultar en la documentación oficial.
- Fuente: <https://docs.python.org/es/3/library/urllib.request.html#module-urllib.request>.
- Las APIs como *REST* utilizan el lenguaje *JSON (JavaScript Object Notation)* que codifica las estructuras de datos de manera tal que sean legibles por las máquinas y puedan ser enviados y recibidos.
- En Python se utilizan mayoritariamente las bibliotecas *requests* para usar *REST* y *json* para convertir datos en ese formato a diccionarios o listas.

```
[ ]: # Instalar la biblioteca Requests
pip install requests
```

- Un ejemplo de uso de *REST* adaptado del tutorial de <https://www.dataquest.io/blog/python-api-tutorial/>.

```
[ ]: import requests
import json
from datetime import datetime

# Módulo REQUESTS
# Ejemplo utilizando la API de Open Notify http://api.open-notify.org/
# Utiliza la función request.get() para conectarse a una API inexistente
response = requests.get("http://api.open-notify.org/esta-api-no-existe")
print(response.status_code)
```

```
[ ]: # Función request.get() con código de estado 200
response = requests.get("http://api.open-notify.org/astros.json")
print(response.status_code)

[ ]: # Función response.json() devuelve datos utilizando la API
print(response.json())
print(type(response.json()))

[ ]: # Obtener datos de iss-now.json
respuesta = requests.get("http://api.open-notify.org/iss-now.json")
print(respuesta.status_code)

def json_print(objeto):
    texto = json.dumps(objeto, sort_keys=True, indent=4)
    print(texto)

json_print(respuesta.json())

[ ]: # Módulo DATETIME
# Extrae los datos de "timestamp"
# Presenta los datos con formato fecha
fecha_unix = respuesta.json()["timestamp"]
json_print(f'Fecha formato Unix: {fecha_unix}')

fecha = datetime.fromtimestamp(fecha_unix)
json_print(f'Fecha formato normal: {fecha}')
```

2.6 Herramientas útiles

- Listado de APIs públicas: <https://github.com/public-apis/public-apis>.
- Listado Rapidapi: <https://rapidapi.com/collection/list-of-free-apis>.
- API marketplace: <https://apilayer.com/>.
- Un simple servicio para HTTP Request & Response: <https://httpbin.org/>.
- Clientes REST/API:
 - <https://insomnia.rest/> Insomnia,
 - <https://www.paw.cloud/> Paw, y
 - <https://www.postman.com/> Postman.
- Para conocer los códigos de errores HTTP se puede visitar la siguiente página web de la Wikipedia:

https://es.wikipedia.org/wiki/Anexo:C%C3%B3digos_de_estado_HTTP.

2.7 Formato YAML

- el lenguaje de serialización de datos YAML (*YAML Ain't Markup Language*) es generalmente utilizado en ficheros de configuración de aplicaciones y sistemas, aunque también se utiliza en almacenamiento de datos (salidas de *debugging*) o transmisión (encabezados de documentos).

- Soporta de forma nativa tres tipos de datos: escalares (como cadenas, enteros y de coma flotante), listas, y matrices.
- A partir de YAML 1.2, el formato se convirtió oficialmente en un superconjunto estricto de JSON, lo que significa que todo documento JSON válido también resulta ser un documento YAML.
- En Python como *parser* se utiliza *PyYAML*. Para instalarlo:

```
[ ]: pip install pyyaml
```

- Fuentes: <https://pyyaml.org/wiki/PyYAMLDocumentation>, <https://pypi.org/project/PyYAML/>, y <https://yaml.org/>.
- Algunos ejemplos del uso de este formato en python:

```
[ ]: import yaml
# Convertir un diccionario a yaml
usuarios = [{'nombre': 'Juan García', 'ocupacion': 'programador'},
            {'nombre': 'María Domínguez', 'ocupacion': 'tester'}]

print(yaml.dump(usuarios, allow_unicode=True))

# Escribir a un fichero
with open('usuarios.yaml', 'wt', encoding='utf-8') as f:
    datos = yaml.dump(usuarios, f, allow_unicode=True)
```

```
[ ]: # Leer ficheros yaml
with open('usuarios.yaml') as f:
    datos = yaml.load(f, Loader=yaml.FullLoader)
    print(datos)
```

- Fuente: <https://realpython.com/python-yaml/>.

2.8 Persistencia de datos

- Los módulos en Python que soportan el almacenamiento de datos de forma persistente en el disco se denominan *pickle* y *marshal*.
- De los dos *pickle* es el recomendado, además de ser el más utilizado.
- Éstos pueden convertir varios tipos de datos de Python en un flujo de bytes y luego recrear los objetos a partir de los mismos.
- Por su parte los módulos relacionados con bases de datos admiten formatos de archivo basados en *hash* que almacenan un mapeo de cadenas a otras cadenas.
- Existe también un módulo llamado *jsonpickle* que permite codificar y decodificar objetos en Python.
- Se recomienda usarlo con cautela ya que puede presentar problemas de seguridad al ejecutar código arbitrario.
- <https://github.com/jsonpickle/jsonpickle>.
- “Serializar” (*pickling*) significa convertir una jerarquía de objetos de Python en una secuencia de bytes,

- “Deserializar” (*unpickling*) es la operación inversa.
- Tener en cuenta que el módulo *pickle* no es seguro, solo se deben “deserializar” datos en los cuales se confía.
- Para datos no seguros el uso de JSON es más apropiado. Fuente: <https://docs.python.org/es/3/library/pickle.html>.
- Algunas diferencias entre JSON y *pickle*:
 - JSON codifica a UTF-8, es legible para seres humanos, interoperable, y representa solo un subconjunto de los tipos integrados de Python.
 - *Pickle* por su parte convierte a código binario, no es legible para el ser humano, es específico de Python y puede representar un número muy grande de tipos de Python.

```
[ ]: # Fichero pickle_pickling.py
import pickle

datos = int(input('Cantidad de datos a ingresar: '))
lista_datos = []

for dato in range(datos):
    datos_in = input('Ingresar dato '+str(dato)+' : ')
    lista_datos.append(datos_in)

with open('pickle_datos', 'wb') as datos:
    pickle.dump(lista_datos, datos)
```

```
[ ]: # Fichero pickle_unpickling.py
import pickle

fichero = open('pickle_datos', 'rb')

datos = pickle.load(fichero)
fichero.close()

print('Datos guardados en el fichero:')

lista = 0
for item in datos:
    print('El dato ', lista, ' es : ', item)
    lista += 1
```

TAREA: Refactorizar el fichero `pickle_unpickling.py`

2.9 Manipular ficheros XML

- *XML (eXtensible Markup Language)* es un lenguaje ampliamente utilizado para almacenar y transportar datos estructurados.
- En Python se utilizan la biblioteca *xml.etree.ElementTree* - *The ElementTree XML API*, o la biblioteca *minidom*, aunque la primera es aconsejable si no se conoce la API del DOM.
- *Minidom (Minimal DOM Implementation)* es una implementación simplificada del *Document Object Model (DOM)*.
- La API del DOM trata al XML como una estructura de árbol donde cada rama o nodo es un objeto.
- https://developer.mozilla.org/es/docs/Web/API/Document_Object_Model/Introduction.
- https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model.
- El siguiente ejemplo de uso de la biblioteca *xml.etree.ElementTree* fue extraído y adaptado de la documentación oficial:
- <https://docs.python.org/es/3.9/library/xml.etree.elementtree.html#module-xml.etree.ElementTree>.

```
[ ]: import xml.etree.ElementTree as ET
# Importar XML desde un fichero
arbol = ET.parse('ejemplo_country_data.xml')
raiz = arbol.getroot()

# Acceder a elementos determinados
for rama in raiz:
    print(rama.tag, rama.attrib)
# Acceder al child utilizando índices
print(raiz[0][1].text)
```

```
[ ]: # Acceder a sub elemento utilizando el método Element
for neighbor in raiz.iter('neighbor'):
    print(neighbor.attrib)
```

```
[ ]: # findall encuentra sólo los elementos con un tag que son hijos directos del
    ↪ elemento actual
for country in raiz.findall('country'):
    rank = country.find('rank').text
    name = country.get('name')
    print(name, rank)
```

```
[ ]: # Modificar un fichero XML y escribirlo a un nuevo fichero
for rank in raiz.iter('rank'):
    new_rank = int(rank.text) + 1
    rank.text = str(new_rank)
    rank.set('Actualizado', 'si')
arbol.write('ejemplo_country_data_modificado.xml')
```

2.10 Seguridad XML

- Ataques a bibliotecas XML pueden ser extremadamente dañinos.
- Los atacantes suelen utilizar funcionalidades poco conocidas de XML.
- La biblioteca *defusedxml* provee varias medidas de contorno para rechazar estos ataques.

```
[ ]: # Instalación
pip install defusedxml
```

- Para utilizarla:

```
[ ]: # Cambiar este código
from xml.etree.ElementTree import parse
et = parse(xmlfile)

# Por este otro:
from defusedxml.ElementTree import parse
et = parse(xmlfile)
```

- Fuente: <https://pypi.org/project/defusedxml/#python-xml-libraries>.

2.11 Descargar ficheros desde Internet

- Para descargar ficheros con Python desde Internet se pueden utilizar entre otras, las bibliotecas *urllib.request*, *wget*, *requests*, y *urllib3*.
- La biblioteca estándar de Python para estas tareas es *urllib* aunque se utilizan mayoritariamente *requests* junto con *urllib3*.
- La biblioteca *urllib* es una colección de varios módulos que permiten trabajar con URLs:
 - *urllib.request* Para abrir y leer URLs.
 - *urllib.error* para gestionar las excepciones recogidas por *urllib.request*.
 - *urllib.parse* para parsear URLs.
 - *urllib.robotparser* para parsear ficheros *robots.txt*.
- Fuente: <https://docs.python.org/3/library/urllib.request.html#module-urllib.request>.

```
[ ]: # Ejemplo descargar y descomprimir un fichero con urllib.request
from io import BytesIO
from urllib.request import urlopen
from zipfile import ZipFile
```

```
[ ]: # Obtener el fichero con los datos de https://insights.stackoverflow.com/survey
# Pertenecen a la encuesta anual de desarrolladores año 2023
zip_url = 'https://cdn.stackoverflow.co/files/jo7n4k8s/production/
          49915bfd46d0902c3564fd9a06b509d08a20488c.zip/
          ↪stack-overflow-developer-survey-2023.zip'
```

```
[ ]: with urlopen(zip_url) as zipresp:
      with ZipFile(BytesIO(zipresp.read())) as zfile:
```

```
zfile.extractall('/home/usuario/')    # Directorio donde guardar los ficheros
```

- *Wget* es una herramienta de los sistemas GNU/Linux y Unix que permite descargar ficheros de Internet.
- El módulo de Python con el mismo nombre se instala con:

```
[ ]: pip install wget
```

```
[ ]: # Ejemplo con wget
import wget

url='https://cdn.stackoverflow.co/files/jo7n4k8s/production/
    49915bfd46d0902c3564fd9a06b509d08a20488c.zip/
    ↪stack-overflow-developer-survey-2023.zip'
wget.download(url)
```

2.12 Biblioteca requests

- La biblioteca *requests* es la recomendada para una interfaz de cliente HTTP de mayor nivel.
- Permite enviar pedidos HTTP/1.1 de manera sencilla.
- Utiliza *urllib3* para mantener activas las conexiones HTTP de forma automática.
- Fuente: <https://requests.readthedocs.io/en/latest/>.

```
[ ]: # Instalación
python -m pip install requests
```

```
[ ]: # Ejemplo descargar y descomprimir un fichero con requests
import requests

url = 'https://cdn.stackoverflow.co/files/jo7n4k8s/production/
    49915bfd46d0902c3564fd9a06b509d08a20488c.zip/
    ↪stack-overflow-developer-survey-2023.zip'
req = requests.get(url)

# Se obtiene solamente el nombre del fichero en la URL
fichero = url.split('/')[-1]

# Descarga el fichero en el directorio1 de la ruta actual
with open('./directorio1/' + fichero, 'wb') as fichero_salida:
    fichero_salida.write(req.content)
```

- *urllib3* proporciona varias características no disponibles en la biblioteca estándar de Python, entre ellas:
 - Seguridad.

- *Pool* de conexiones.
- Verificación SSL/TLS del lado del cliente.
- Cargas de ficheros con codificación multiparte.
- Asistentes para reintentar solicitudes y gestionar redireccionamientos HTTP.
- Compatibilidad con la codificación gzip, deflate, brotli y zstd.
- Compatibilidad con proxy para HTTP y SOCKS.

- Fuente: <https://github.com/urllib3/urllib3>.

```
[ ]: # Instalación
python -m pip install urllib3
```

2.13 BeautifulSoup

- *BeautifulSoup* es un parser o analizador sintáctico HTML ideal para realizar *web scraping*.
- Permite abrir páginas web y extraer datos estructurados a partir de patrones para luego manipularlos.
- El web scraping con BeautifulSoup tiene la siguiente estructura:
 - URL → Solicitud HTTP → HTML → BeautifulSoup

```
[ ]: pip install beautifulsoup4 # Instalación de la biblioteca
pip install lxml # Instalación del parser a utilizar con la biblioteca
```

```
[ ]: # Ejemplo uso de la biblioteca Beautiful Soup
# Documentación oficial:
# https://www.crummy.com/software/BeautifulSoup/bs4/doc/
from bs4 import BeautifulSoup
import requests

url = input("Escribe la URL del sitio web: ")
agrega_http = requests.get("http://" + url)
datos = agrega_http.text

soup = BeautifulSoup(datos, features="html5lib")

for link in soup.find_all("a"):
    print(link.get("href"))
```

```
[ ]: # Ejemplo extraído de https://scipython.com/blog/
↳scraping-a-wikipedia-table-with-beautiful-soup/
import urllib.request
from bs4 import BeautifulSoup

url = 'https://en.wikipedia.org/wiki/ISO_3166-1_alpha-2'
req = urllib.request.urlopen(url)
article = req.read().decode()
```

```

with open('ISO_3166-1_alpha-2.html', 'w') as fo:
    fo.write(article)

# Load article, turn into soup and get the <table>s.
article = open('ISO_3166-1_alpha-2.html', encoding='UTF-8').read()
soup = BeautifulSoup(article, 'html.parser')
tables = soup.find_all('table', class_='sortable')

# Search through the tables for the one with the headings we want.
for table in tables:
    ths = table.find_all('th')
    headings = [th.text.strip() for th in ths]
    if headings[:5] == ['Code', 'Country name', 'Year', 'ccTLD', 'ISO 3166-2']:
        break

# Extract the columns we want and write to a semicolon-delimited text file.
with open('iso_3166-1_alpha-2_codes.txt', 'w') as fo:
    for tr in table.find_all('tr'):
        tds = tr.find_all('td')
        if not tds:
            continue
        code, country, year, ccTLD = [td.text.strip() for td in tds[:4]]
        # Wikipedia does something funny with country names containing
        # accented characters: extract the correct string form.
        if '!' in country:
            country = country[country.index('!')+1:]
        print('; '.join([code, country, year, ccTLD]), file=fo)

```

2.14 Scrapy

- Otra herramienta de scraping para Python es Scrapy <https://scrapy.org/>:
 - Servicio <https://www.zyte.com/scrapy-cloud/>.
 - Scrapy spiders <https://scrapy.readthedocs.io/en/stable/>.

```
[ ]: pip install scrapy
```

```

[ ]: # Ejemplo extraído de https://scrapy.org/
# Fichero mi_spider.py
import scrapy

class BlogSpider(scrapy.Spider):
    name = 'blogspider'
    start_urls = ['https://www.zyte.com/blog/']

    def parse(self, response):
        for title in response.css('.oxy-post-title'):

```

```

        yield {'title': title.css('::text').get()}

    for next_page in response.css('a.next'):
        yield response.follow(next_page, self.parse)

```

- Se ejecuta el fichero con la orden:

```
[ ]: scrapy runspider mi_spider.py
```

2.15 Selenium

- Selenium facilita una API para poder escribir tests de automatización de aplicaciones web.
- También permite automatizar tareas de administración web.
- Tiene soporte para los navegadores más utilizados actualmente como Firefox, Chrome, Edge, y Safari.
- Fuente: <https://www.selenium.dev/>.
- Documentación Selenium: <https://selenium-python.readthedocs.io/>.
- Se instala con:

```
[ ]: pip install selenium
```

- Para poder hacer las pruebas se deberán instalar los controladores para los navegadores web.
- https://www.selenium.dev/documentation/webdriver/getting_started/install_drivers/.

```
[ ]: # Ejemplo: Se instala el controlador para Firefox en sistemas GNU/Linux Debian/
↳ Ubuntu
sudo apt install firefox-geckodriver
```

```
[ ]: # Ejemplo básico de automatización
from selenium import webdriver

probar = webdriver.Firefox()
probar.get('https://fortinux.com')
print('Título de la página: %s', probar.title)
print(probar.page_source)
probar.quit()
```

2.16 Errores y excepciones en Python

- En python dentro de los diferentes tipos de errores se pueden mencionar los errores de sintaxis y las excepciones.
- Todas las excepciones deben ser instancias de una clase que se derive de *BaseException*.
- Las nuevas excepciones se recomienda que deriven a partir de la clase *Exception*.
- Las excepciones en Python se manejan esencialmente con *try-except*.

- Documentación oficial: <https://docs.python.org/es/3/tutorial/errors.html>.
- Excepciones incorporadas: <https://docs.python.org/es/3/library/exceptions.html>.
- Definidas por el usuario <https://docs.python.org/es/3/tutorial/errors.html#tut-userexceptions>.

2.17 Errores de sintaxis

- Los errores de sintaxis se muestran cuando Python no consigue interpretar el código debido a una sintaxis errónea.

```
[2]: >>> print ('Hola mundo')
File "<stdin>", line 1
      print ('Hola mundo')
```

```
File "/tmp/ipykernel_783/2391534570.py", line 1
      print ('Hola mundo')
      ^
SyntaxError: EOL while scanning string literal
```

2.18 Excepciones

- Los errores en Python detectados durante la ejecución se llaman excepciones.
- El bloque *try* prueba un bloque de código en busca de errores.
- El bloque *except* gestiona los errores.
- El bloque *finally* permite ejecutar código, a pesar de los resultados de *try - except*.
- `ZeroDivisionError`: División en la que el divisor es cero o no se puede distinguir de cero.

```
[ ]: def funcion_excepciones():
      try:
          # Dividir entre cero genera una excepción
          print(10 / 0)
      except ZeroDivisionError:
          print("Error. No se puede dividir por cero.")
      else:
          # La excepción no ha ocurrido
          print("La excepción no ha ocurrido")
      finally:
          # Este bloque se ejecuta cuando todas
          # las excepciones han sido ejecutadas
          print("Finalizadas las excepciones.")

funcion_excepciones()
```

- `ValueError`: Valores inapropiados. Cuando una función (como `int()` o `float()`) recibe un argumento de un tipo adecuado, pero su valor es inaceptable.

```
[ ]: import math
import warnings
warnings.filterwarnings("ignore")

try:
    numero = int(input('Escribe un número positivo: '))
    try:
        print(f'La raíz cuadrada de {numero} es {math.sqrt(numero)}')
    except ValueError as ve:
        print(f'{numero} no es un número positivo.')
except ValueError as ve:
    print("Debes escribir un número entero.")
```

- TypeError: Dato con un tipo inadecuado.

```
[1]: lista = [5, 3, 4]
valor = lista[0.5]
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[1], line 2
      1 lista = [5, 3, 4]
----> 2 valor = lista[0.5]

TypeError: list indices must be integers or slices, not float
```

- SyntaxError: Cuando una línea de código viola la gramática de Python.

```
[2]: def funcion()
      return
```

```
Cell In[2], line 1
      def funcion()
      ~
SyntaxError: expected ':'
```

- AttributeError: Cuando se intenta utilizar un método que no existe para ese elemento.

```
[3]: lista = [5]
lista.depend(2)
```

```
-----
AttributeError                            Traceback (most recent call last)
Cell In[3], line 2
      1 lista = [5]
```

```
----> 2 lista.depend(2)
```

```
AttributeError: 'list' object has no attribute 'depend'
```

- `NameError`: Cuando no se encuentra un nombre local o global.

```
[4]: 5 * elemento
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[4], line 1  
----> 1 5 * elemento  
  
NameError: name 'elemento' is not defined
```

- Por último *raise* permite forzar a que ocurra una excepción:

```
[5]: >>> raise NameError("Hola Mundo")
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[5], line 1  
----> 1 raise NameError("Hola Mundo")  
  
NameError: Hola Mundo
```

```
[3]: variable = "1"  
  
if not type(variable) is int:  
    raise TypeError("Solamente se admiten números enteros")
```

```
-----  
TypeError                                Traceback (most recent call last)  
/tmp/ipykernel_452/1387624469.py in <module>  
      2  
      3 if not type(variable) is int:  
----> 4     raise TypeError("Solamente se admiten números enteros")  
  
TypeError: Solamente se admiten números enteros
```

2.19 Depuración de errores

- El proceso durante el cual se eliminan los errores del código se llama depuración.
- Una técnica de depuración simple es la conocida como la depuración por impresión:
 - Se inserta *print()* dentro del código para verificarlo

- Por ej. imprimir valores de variables.
- Otra técnica es el uso de *assertions* en Python.
- La palabra reservada *assert* muestra un error cuando el enunciado es falso.
- Se puede probar por ejemplo si un argumento es *not None* o si el valor de retorno es una cadena de texto.
- Sirve además para generar casos de pruebas (*test cases*) de software.

```
[6]: >>> assert 1 == 0
```

```
-----
AssertionError                                Traceback (most recent call last)
Cell In[6], line 1
----> 1 assert 1 == 0

AssertionError:
```

- Un ejemplo utilizando la biblioteca *sys*:

```
[7]: import sys
assert ('linux' in sys.platform), "solamente se ejecuta en sistemas Linux"
assert ('win32' in sys.platform), "solamente se ejecuta en sistemas Windows"
```

```
-----
AssertionError                                Traceback (most recent call last)
Cell In[7], line 3
      1 import sys
      2 assert ('linux' in sys.platform), "solamente se ejecuta en sistemas_
↳Linux"
----> 3 assert ('win32' in sys.platform), "solamente se ejecuta en sistemas_
↳Windows"

AssertionError: solamente se ejecuta en sistemas Windows
```

- Esta técnica se usa también para crear la documentación del código ya que permite realizar alguna acción, al contrario de los *docstrings* y comentarios.

2.20 Logging

- El módulo *logging* contiene un conjunto de funciones que permiten registrar los eventos del programa.
- Ellas son: *debug()*, *info()*, *warning()*, *error()* y *critical()*.
- Un ejemplo básico:

```
[ ]: import logging
```

```
# Muestra un mensaje en la consola:q

logging.warning('Atención!')
# No muestra ningún mensaje
logging.info('Es solo una información')
```

- Para enviar la información de los eventos a un fichero:

```
[ ]: import logging

logging.basicConfig(filename='ejemplo.log', format='%(asctime)s %(message)s',
                    encoding='utf-8', level=logging.DEBUG)
logging.debug('Este es el mensaje de DEBUG')
logging.info('Esta es una información')
logging.warning('Aquí está la advertencia')
logging.error('Mensajes de ERROR')
```

- Otro ejemplo del uso de *logging*:

```
[ ]: import logging
logging.basicConfig(filename='./errores.log', level=logging.DEBUG,
                    format='%(asctime)s %(levelname)s %(name)s %(message)s')
logger=logging.getLogger(__name__)

try:
    1/0
except ZeroDivisionError as err:
    logger.error(err)
```

- Fuente: <https://docs.python.org/es/3/howto/logging.html>.
- Tutorial: <https://realpython.com/python-logging/>.

2.21 Formatos de docstrings

- Existen formatos específicos de *docstrings* que ayudan a los usuarios y a los *parsers* de Python con la documentación.
- Los más utilizados son:
 - numpydoc: el más amigable al usuario. <https://numpydoc.readthedocs.io/en/latest/format.html>.
 - reStructuredText: el utilizado por Python <https://docutils.sourceforge.io/rst.html>.
 - Ejemplo: <https://docutils.sourceforge.io/docutils/statemachine.py>.
 - Google Style Guide <https://google.github.io/styleguide/pyguide.html#38-comments-and-docstrings>.
- El ejemplo adaptado de numpydoc <https://numpydoc.readthedocs.io/en/latest/example.html#example>:

```
[ ]: vim docs/numpydoc_ejemplo.py
```

```
[ ]: """
Docstring del modulo ejemplo_numpydoc

(cuerpo del docstring) Docstrings son comentarios
de varias líneas de texto que se insertan antes de
un módulo, clase, etc. explicando su función.
Adaptado de:
https://numpydoc.readthedocs.io/en/latest/example.html#example

Author: Fortinux.
"""

def foo(var1, var2, *args, nombre_largo_var="hola",
        una_keyword_usada_raramente=0, **kwargs):
    r"""Sumario de la función en una línea.

    La descripción completa.
    En varias líneas
    Las variables usan back-ticks: `var`.

    Parametros
    -----
    var1 : variable tipo matriz
        Esto significa -- lists, nested lists, etc. --
    var2 : int
        Tipo entero en Python
    *args : iterable
        Otros argumentos.
    nombre_largo_variable : {'hoola', 'hola!'}, optional
        Opciones en llaves, si es opcional, primero la default.

    Returns
    -----
    type
        Explicación del valor de retorno de tipo ``type``.

    Notas
    -----
    Notas si son necesarias.
    .. math:: X(e^{j\omega}) = x(n)e^{-j\omega n}
    Símbolo griego :math:`\omega` en una línea.
```

Referencias

Ejemplo de cita: [1]_. Se pueden incluir las citas también en la sección notas.

.. [1] O. McNoleg, "The integration of GIS, remote sensing, expert systems and adaptive co-kriging for environmental habitat modelling of the Highland Haggis using object-oriented, fuzzy-logic and neural-network techniques," *Computers & Geosciences*, vol. 22, pp. 585-588, 1996.

Ejemplos

Escritos en formato doctest, deben explicar como usar la función.

```
>>> a = [1, 2, 3]
>>> print([x + 3 for x in a])
[4, 5, 6]
>>> print("a\nb")
a
b
"""
pass
```

- se puede en el mismo documento imprimir esta ayuda:

```
[ ]: print(foo.__doc__)
```

```
[ ]: >>> import docs.numpydoc_ejemplo
/root
>>> help(docs.numpydoc_ejemplo)
```

2.22 Doctests

- La librería estándar de Python contiene un módulo llamado *doctest* que permite realizar la documentación y los casos de prueba simultáneamente.
- El módulo contiene un *tests framework* con una leve curva de aprendizaje, lo que permite automatizar los procesos de pruebas inmediatamente.

```
[ ]: python -m doctest docs/numpydoc_ejemplo.py
```

- Fuente y un completo tutorial sobre cómo usarlo:
- <https://realpython.com/python-doctest/>.

2.23 Pruebas unitarias

- Otra técnica es la llamada de pruebas unitarias.

- Python proporciona un módulo llamado *unittest*. <https://docs.python.org/es/3.10/library/unittest.html>
- El framework *pytest* <https://docs.pytest.org/en/latest/> hace fácil escribir pequeñas pruebas, además de poder escalar y soportar complejas pruebas funcionales para aplicaciones y bibliotecas.
- Un tutorial se puede encontrar en <https://fortinux.com/tutoriales/realizar-pruebas-codigo-python/>

2.24 Argparse

- Python utiliza la biblioteca *argparse* (*argument parse*) para programar una interfaz utilizable en la línea de comando.
- Esta biblioteca permite usar argumentos de posición, personalizar caracteres, y utilizar sub-comandos entre otras cosas.
- La documentación se encuentra en: <https://docs.python.org/es/3/library/argparse.html>.
- El tutorial de la misma: <https://docs.python.org/es/3/howto/argparse.html#id1>.
- En la línea de comando se trabaja con argumentos separados por espacios que contienen, como en el siguiente ejemplo, un comando (*ls*), sus argumentos (*-la*), y parámetros (*/etc*):

```
[ ]: ls -la /etc
```

```
[ ]: # Fichero ejemplo_argparse.py
import argparse
import os
import sys

""" Crea el parser """
mi_parser = argparse.ArgumentParser(prog='ejemplo_argparse.py',
                                    description='Listado del directorio',
                                    epilog='Muchas gracias.')

""" Agrega los argumentos """
mi_parser.add_argument('Path',
                      metavar='Ruta',
                      type=str,
                      help='La ruta al directorio')
```

```
[ ]: """ Ejecuta el método parse_args() """
args = mi_parser.parse_args()

dir_ruta = args.Path

if not os.path.isdir(dir_ruta):
    print('Este directorio no existe')
```

```
sys.exit()
```

```
print('\n'.join(os.listdir(dir_ruta)))
```

```
[ ]: python ejemplo_argparse.py directorio  
python ejemplo_argparse.py -h
```